Comparative Analysis of Machine Learning and Large Language Models for Programming Error Pattern Recognition in Competitive Programming

WENBIN HU, School of Computer Science, Xijing University, China

Context: Automated programming-error diagnosis is valuable for computing-education platforms, yet most approaches require source-code access or costly LLM inference. Competitive programming platforms return both a structured error verdict and rich execution metadata for every submission, raising the question of precisely where metadata becomes sufficient and where it reaches its limits.
Objective: We ask the question that prior work has not answered: *At exactly which error-type boundary does metadata become insufficient?* We systematically compare traditional ML and LLMs to establish the metadata ceiling and identify the educationally most consequential boundary—Wrong Answer versus Runtime Error—where source-code integration becomes necessary.
Method: 13,360 Codeforces submissions across five verdict types. We evaluate Random Forest, Gradient Boosting, and Logistic Regression (7 metadata features) plus two LLM configurations (DeepSeek-V3 and Qwen2.5:3B, using the 5 metadata features available without source code), under identical zero-shot protocols. Significance is assessed with McNemar's test; we establish the metadata ceiling for each verdict class individually.
Results: Metadata is sufficient for three of five verdict types: Gradient Boosting recovers Compilation Error, Time Limit Exceeded, and Memory Limit Exceeded with high reliability (F1 $\geq$0.89) by reading the judge's own resource-limit signals. The scientifically and educationally interesting boundary—Wrong Answer versus Runtime Error, where a teacher must distinguish a logic error from a runtime failure—is not separable from metadata alone (RE F1 = 0.52; all models, including frontier LLMs, fail here). The two LLMs differ sharply: DeepSeek-V3 reaches 76.65% on valid predictions (0.8% invalid) but collapses on RE (F1 = 0.03), while Qwen2.5:3B achieves only 35.50% (10.1% invalid). Neither LLM closes the gap to ML (95.02%); the DeepSeek-V3 gap (15.4pp below 5-feature GB) establishes a realistic ceiling for current online LLMs on this task.
Conclusions: We introduce and empirically validate the **metadata ceiling**—the theoretical upper bound on error classification accuracy using only submission metadata. Three verdict types (CE, TLE, MLE) are near-deterministically recoverable (ROC-AUC $\geq$ 0.98); the WA$\leftrightarrow$RE boundary is not (RE F1 = 0.52), and this is precisely where targeted pedagogical feedback requires source-code analysis. This ceiling informs the design of automated feedback systems and defines the fundamental limit of metadata-only classification in programming education. We release the dataset, pipeline, and actionable deployment guidelines for privacy-preserving educational analytics.

CCS Concepts: • **Social and professional topics** → **Computing education**.

Additional Key Words and Phrases: programming error classification, machine learning, large language models, competitive programming, Codeforces

Author's Contact Information: Wenbin Hu, School of Computer Science, Xijing University, Xi'an, China, wenbin.hu2026@outlook.com.

ACM 1946-6226/2026/7-ART1
https://doi.org/10.1145/nnnnnnn.nnnnnnn

ACM Reference Format:
Wenbin Hu. 2026. Comparative Analysis of Machine Learning and Large Language Models for Programming Error Pattern Recognition in Competitive Programming. *ACM Trans. Comput. Educ.* 1, 1, Article 1 (July 2026), 30 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Introductory programming courses report 30–40% dropout rates [10, 53]. Debugging difficulty is a primary obstacle [53], as novices struggle to distinguish error types—misinterpreting runtime errors as logical errors [50].

Prior work analyzed compilation behavior [25] and scaled to thousands of users via Blackbox [12], revealing systematic debugging differences between student groups.

The CS1QA dataset[32] provided a rich resource of question–code pairs from introductory programming courses at a large public university, enabling deeper analysis of student learning patterns through natural language questions accompanying code submissions. Concurrently, competitive programming platforms such as Codeforces, LeetCode, and AtCoder have emerged as valuable environments for studying programming behavior at scale. Unlike classroom settings, competitive programming platforms provide standardized, structured feedback for every submission: execution time measured in milliseconds, memory consumption in kilobytes, precise error verdicts (WA, TLE, RE, CE, MLE), and the number of test cases passed before failure. This structured metadata makes these platforms ideal sources for automated error classification research, while also representing an ecologically valid environment where participants are intrinsically motivated to produce correct and efficient code.

A central methodological point shapes this work. For three of the five verdicts, the verdict is *partly determined by* the execution metrics themselves: a Compilation Error executes in (near) zero time, a Time Limit Exceeded submission runs to the time limit, and a Memory Limit Exceeded submission exhausts the memory limit. A metadata classifier therefore recovers these three verdicts in part because the judge's own measurements encode them. The scientifically interesting question is consequently *not* whether a classifier can reproduce the verdict, but rather *how much error-type information is genuinely recoverable from metadata alone, and precisely where metadata becomes insufficient.* We term this boundary the **metadata ceiling**: the maximum classification accuracy achievable using only submission metadata (problem rating, language, execution time, memory consumption, test cases passed) without source-code access. The residual boundary—distinguishing a Wrong Answer from a Runtime Error—is the educationally meaningful case and the one on which execution metrics provide little traction. Our study is organized around this decomposition.

Caveat on population generalizability: Our dataset comes from competitive programmers on Codeforces (rating 800–3,500). Error patterns in CS1/CS2 student populations differ substantially: novices produce more syntax errors and fewer complex runtime errors, and the competitive programming context (time pressure, algorithmic challenge) may not map directly to classroom learning goals. We evaluate generalizability via a skill-tier analysis (Section 4.7), but validation on student datasets (CS1QA[32], Blackbox) is needed before drawing conclusions for novice programming education contexts.

LLMs (GPT-4, Claude, DeepSeek-Coder) show strong code capabilities [19, 47] but their effectiveness for metadata-only error classification is understudied. LLM error messages often fail to meet formative feedback criteria [34, 40], motivating controlled ML-LLM comparison.

Research Gap and Novelty

While both ML and LLM approaches have been applied to programming error analysis, *no prior work has systematically compared them on the same dataset using identical evaluation protocols.* Prior comparative studies either (1) evaluate only ML methods, (2) evaluate only LLM methods [19, 47], or (3) use different datasets and evaluation metrics, making direct comparison impossible.

Furthermore, existing work predominantly relies on *source-code analysis* (ASTs, token sequences, code embeddings) [18], which incurs high computational cost, requires source code access (raising privacy concerns), and cannot scale to platform-level deployment where millions of submissions must be processed daily.

This paper addresses both gaps. We provide a controlled ML–LLM comparison on the *same* dataset with *identical* evaluation protocol, and we characterize what submission metadata alone can and cannot reveal about error type. Our results show that metadata-based classification recovers the three execution-bounded verdicts reliably and at low cost, whereas the WA↔RE boundary requires source-code analysis. These findings have practical implications for educational platforms seeking to deploy error classification at scale.

To address this gap, we formulate the following three research questions:

- **RQ1:** What are the most prevalent programming error patterns in competitive programming submissions, and how do they distribute across different error categories?
- **RQ2:** How do traditional ML classifiers perform on this task, which features are most predictive, and how stable are predictions under cross-validation?
- **RQ3:** Where exactly does metadata become insufficient for error classification—i.e., can ML and LLMs distinguish Wrong Answer from Runtime Error—and which approach provides the most cost-effective path toward targeted feedback in educational settings?

Our contributions are:

(1) **Controlled comparative study** of ML and LLM methods for error type classification from submission metadata, spanning five experimental conditions (three traditional ML, two LLM-based) under a consistent evaluation protocol.
(2) **Decomposition of metadata signal**: we show that the three execution-bounded verdicts (CE, TLE, MLE) are near-deterministically recoverable from execution metrics, while the WA↔RE distinction—the educationally meaningful case—is not separable from metadata alone (RE F1 = 0.52), precisely demarcating where source-code analysis becomes necessary.
(3) **Comprehensive statistical analysis** including feature importance quantification via Gini importance, systematic ablation studies with McNemar significance testing, confusion matrix analysis with class-wise F1 scores, and power analysis for sample size justification.
(4) **Execution-metric dominance finding**: execution time and memory are the dominant predictors (8.8 pp drop on time removal, $p < 0.001$), reflecting that runtime metrics directly encode the three execution-bounded verdicts; problem-level features contribute only marginally, confirming the source of the metadata signal.
(5) **Actionable method-selection guidelines** that balance accuracy, cost, latency, and explainability across six distinct educational deployment contexts, supported by empirical evidence.
(6) **Open framework** with publicly released dataset, preprocessing pipeline, and experiment code to facilitate replication and extension by the research community.

2 Related Work

2.1 Programming Error Analysis

The study of programming errors has a rich history spanning over four decades in computing education research [55]. This psychological perspective on programming pedagogy highlights the cognitive demands debugging places on novices [55]. McCauley et al.[37] conducted a comprehensive review of the novice programmer literature, identifying syntax errors, logical errors, and conceptual misunderstandings as the three dominant error categories. Their meta-analysis revealed that novices spend approximately 25% of their programming time on debugging activities, yet novice debugging effectiveness improves only modestly over a semester of instruction. This finding underscores the need for automated tools that can accelerate the development of debugging proficiency. Cognitive Load Theory [51] posits that debugging simultaneously taxes working memory across error diagnosis, code comprehension, and solution planning, creating intrinsic overload that constrains novice self-correction [30]. Jadud

Jadud [25] pioneered compilation event analysis using the BlueJ IDE, establishing compilation behavior as a rich data source for educational mining. The Blackbox project [12] scaled this to 200,000+ users, revealing that compilation errors cluster early in programming episodes and that high-performing students compile more strategically.

Large-scale analyses [1, 36] established that runtime errors persist across cohorts and resist standard interventions, motivating automated diagnostic tools [7].

Spohrer and Soloway [50] provided an influential early theoretical framework distinguishing between plan composition errors (faulty algorithm design or implementation strategy) and plan comprehension errors (misunderstanding of specific language constructs). Their controlled studies with novice LISP programmers demonstrated that many seemingly simple errors have deeper cognitive roots than syntax misunderstanding, indicating that effective automated feedback must identify the underlying error pattern. Students' self-efficacy beliefs compound this challenge: Gorson and O'Rourke [21] found that CS1 students systematically overestimate their error severity, often attributing correct execution to luck rather than understanding—a metacognitive gap that automated tools could help correct.

Busjahn et al. [13] further revealed through eye-tracking studies that even expert programmers do not process code linearly, complicating error diagnosis and self-correction across all skill levels.

2.2 Machine Learning in Computing Education

Machine learning has been increasingly applied to computing education challenges over the past fifteen years. Educational data mining has identified programming education as a key application domain [5, 48], with prediction tasks dominating the literature. Metadata-based approaches achieve strong performance [4, 25], while deep learning enables knowledge tracing in code-based learning [44].

Behavioral features from early assignments—compilation frequency, error diversity, time-on-task—can predict course outcomes [12, 25], enabling early intervention through instructor-facing dashboards [26, 52].

Robust behavioral modeling distinguishes productive from unproductive debugging [17], aligning with cognitive load principles [51]. Process mining approaches [48] highlight the value of analyzing interaction sequences over aggregated statistics.

Prior work on error classification from submission data has shown that metadata features can achieve strong performance when combined with source-code analysis. Yet the computational cost of extracting code-level features (ASTs, cyclomatic complexity, token diversity) limits scalability for platform-level deployment. Our approach demonstrates that metadata-only features—without source code access—can achieve competitive accuracy, offering a practical alternative for large-scale deployment. Allamanis et al. [2] demonstrated that graph-based representations of program structure capture semantically meaningful features for code-related prediction tasks, motivating the incorporation of such learned representations into error classification.

Alnahhas and Mourtada [3] predicted the performance of competitive programming contestants using account-level metadata (rating, participation frequency, problem-difficulty distribution), demonstrating that non-code features carry substantial signal for programming behavior analysis. While their focus was on performance prediction rather than error classification, their work demonstrates the continued relevance of simple, interpretable models in educational contexts.

2.3 Large Language Models in Programming

Large language models have rapidly transformed expectations for automated code understanding and generation [8]. Barke et al. [8] demonstrated that programmer interaction with code-generating models differs fundamentally from traditional IDE usage, with implications for how error feedback should be designed in AI-augmented environments. Finnie-Ansley et al. [19] conducted an early evaluation of GPT-3 on CS1 examination questions, finding that while the model correctly answered only 56% of the questions, it produced plausible yet often incorrect explanations for wrong answers, raising concerns about over-reliance on LLM outputs in educational settings.

Prior work has established that LLMs excel at code generation and repair [28, 49]. Sarsa et al.
citesarsa2022automatic demonstrated that LLM-based feedback generation achieves expert-level quality [35]; Leinonen et al. [35] specifically demonstrated that LLM-enhanced error messages measurably improved novice self-correction rates. Burrows et al.
citeburrows2014eras established NLP frameworks for evaluating generated feedback quality. Yet our Qwen2.5:3B zero-shot results (31.92% accuracy; 10.1% invalid outputs) demonstrate that fine-grained error-type classification remains a distinct challenge, a gap also noted by Finnie-Ansley et al. [19] regarding LLM limitations in code analysis. Katic et al. [47] demonstrated that GPT-4 can provide useful debugging explanations for CS1 student code, though quality varied considerably across problem types.

Feng et al. [18] showed that CodeBERT—a pre-trained transformer model for code—achieves state-of-the-art performance on multiple code classification benchmarks including code search, clone detection, and defect detection. CodeBERT's advantage lies in its ability to learn rich code representations from large-scale unlabeled code corpora through masked language modeling objectives. In practice, CodeBERT has primarily been evaluated on code-level features (tokens, ASTs) rather than metadata-based classification, and its effectiveness on submission metadata alone remains unclear.

Recent studies [19, 28] conducted a large-scale study of GPT-3.5 and GPT-4 as programming assistants for CS1 assignments, finding that LLMs could solve 80%–92% of typical programming assignments but struggled with nuanced debugging tasks requiring understanding of the student's intent versus the implemented code. This underscores a limitation: LLMs excel at generating correct solutions but face challenges in analysis tasks requiring comparison against intended behavior.

Kazemitabaar et al. [28] found that while LLM-generated explanations were rated as clearer than human-generated ones, they were less accurate for non-obvious errors, with many explanations failing to correctly identify the root cause for runtime errors. This accuracy gap mirrors our own finding that RE remains the most challenging category even for state-of-the-art LLMs.

Recent work by Leerentveld et al. [34] found that LLM-enhanced programming error messages—generated by GPT-4 and similar models—were "ineffective in practice" in a controlled classroom study: students receiving LLM-generated explanations did not debug more effectively than those receiving standard compiler messages. This finding aligns with the performance gap we observe in our controlled experiments.

Lee et al. [33] examined LLM-generated explanations of compiler errors and found that while the explanations were linguistically fluent, they often contained hallucinated details about code behavior, particularly for runtime errors where the model had to infer execution paths without actually running the code. This limitation is directly relevant to our metadata-only classification setting, where Qwen2.5:3B also struggled with RE classification.

Messeri et al. [46] systematically benchmarked multiple LLMs (GPT-4, PaLM-2, LLaMA-2) on code understanding tasks, reporting that all models exhibited significant performance degradation on error identification tasks compared to code generation tasks. Their results indicate that current LLMs are better suited as code generators than as error diagnosticians, motivating specialized error classification approaches. Recent studies have examined how students learn to program alongside AI as a learning partner in secondary education [28]; Santos et al. Prather et al. [45] conducted a comprehensive review of how generative AI tools are reshaping computing education, identifying both opportunities and risks for novice programmers. . [9] examined assessment outcomes in CS1 courses affected by generative AI tools, providing empirical evidence for the broader impact of LLMs on programming education.

Recent work has explored hybrid approaches that combine ML classification with LLM explanation generation, proposing two-stage architectures where ML handles initial error classification and LLMs provide natural language explanations. Though preliminary, these approaches have been evaluated primarily on synthetic datasets rather than real competitive programming submissions. Our work extends this line by providing the first comprehensive ML–LLM comparison on real-world competitive programming data. Keuning et al. [29] provide a TOCE systematic review of AI for programming education. Recent work has surveyed large language models in education broadly [19, 28], evaluated LLM-generated code quality in CS assignments [46], and examined student perceptions of AI as a learning partner [9, 16], informing the design of hybrid systems.

2.4 Metadata-Based Classification in Educational Contexts

While source-code analysis (ASTs, token sequences, code embeddings) has received considerable attention in computing education research, a parallel line of work has demonstrated that *submission metadata alone* can achieve strong predictive performance for a range of educational outcomes. Jadud [25] showed that compilation frequency and error persistence patterns extracted from IDE interaction logs can predict course outcomes with moderate accuracy ($R^2 \approx 0.3$). The Blackbox project [12] scaled this approach to over 200,000 students worldwide, demonstrating that behavioral metadata (compilation sequences, time-on-task, submission frequency) constitute a rich and continuously available signal for educational data mining.

Prior work has shown that features derived from early programming submissions—including compilation frequency, error diversity, and time-on-task—can predict final course grades using only metadata, without accessing student source code [12, 25]. This established that *behavioral signals* from the first four weeks of a semester are sufficient to identify students at risk of failure, enabling early intervention without privacy-invasive code monitoring.

In the context of competitive programming platforms, Alnahhas and Mourtada [3] used account-level metadata (contestant rating, participation frequency, problem difficulty distribution) to predict overall competitive programming performance, reporting 78% accuracy using only non-code features. Their findings indicate that metadata alone carries substantial signal for programming behavior analysis, motivating our focus on metadata-based error classification.

The advantages of metadata-based classification are both practical and privacy-preserving: (1) no source code access is required, avoiding potential concerns about student code being processed by third-party APIs; (2) computational cost is orders of magnitude lower than code-based approaches (no tokenization, AST parsing, or GPU inference); and (3) deployment at platform scale (millions of daily submissions) becomes feasible with modest computational resources. A central open question is whether metadata alone can achieve competitive accuracy against code-based approaches—a question our work is, to our knowledge, the first to systematically address for the error classification task.

Contribution boundary: Prior work on metadata-based prediction has focused primarily on *grade prediction* and *at-risk identification* [42]. We are not aware of any prior study that has applied metadata-based classification to the more granular task of *error type classification* across five error categories, nor has any work systematically compared metadata-based ML against LLM-based approaches on the same task.

No prior study has systematically compared traditional ML and LLM approaches on the same programming error classification task using the same dataset and evaluation protocol. We identify three specific gaps:

(1) **Metadata-based classification**: Existing error classification studies focus on source code analysis, neglecting the rich diagnostic signals in submission metadata (execution time, memory usage, test cases passed).
(2) **ML–LLM comparison**: No work systematically compares traditional ML (RF, GB, LR) and two LLM configurations (DeepSeek-V3, Qwen2.5:3B) on identical error classification tasks.
(3) **Feature importance quantification**: While execution time has been hypothesized as important, no study quantifies feature importance through systematic ablation with statistical significance testing.

Our work addresses all three gaps by providing the first comprehensive comparison of ML and LLM methods for metadata-based error classification, with systematic feature ablation and McNemar significance testing.

3 Methodology

3.1 Dataset Collection

We collected submission data from Codeforces, a competitive programming platform widely used for algorithmic training and education [39, 57] (https://codeforces.com/) via the official public REST API. The Codeforces platform was selected for three reasons: (1) it provides structured, rich metadata for every submission including precise execution time, memory usage, and error verdict; (2) it represents an ecologically valid competitive programming environment with high participant engagement; and (3) it has a large, diverse problem set covering a wide range of algorithmic domains and difficulty levels.

Submissions were retrieved from competitive programmers with Codeforces ratings spanning 800 to 3500 (corresponding to Newbie through Legendary Grandmaster levels). This range was deliberately chosen to represent proficient programmers who produce a variety of error types and for whom submission metadata is meaningfully differentiated. Data collection was completed in June 2026, covering submissions from November 2020 to June 2026, providing approximately 5.5 years of programming activity.

Collection Pipeline. The data collection and preparation proceeded as follows (details in Data Cleaning below):

(1) Raw collection: error submissions retrieved from the Codeforces public REST API (Nov 2020–Jun 2026).
(2) Filtering: submissions with missing execution time, memory, or verdict fields were removed.
(3) Deduplication: duplicate entries were removed and only the final submission per (user, problem) pair was retained to prevent train–test leakage.
(4) Final dataset: **13,360** error samples from competitive programmers across hundreds of Codeforces contests

Data Cleaning. Data cleaning was performed in three stages. First, submissions with incomplete metadata (missing execution time, memory, or verdict fields) were removed. Second, duplicate entries (identical submission timestamps and verdicts) were deduplicated. Third, to prevent data leakage between training and test sets, only the final submission per (user, problem) pair was retained—this ensures that no two submissions in the dataset are from the same user solving the same problem, preventing the model from learning user-specific artifact patterns.

The final dataset comprises 13,360 error submissions from competitive programmers across hundreds of Codeforces contests. Table 1 summarizes the error type distribution. The language distribution is predominantly C++ (C++23 41.1%, C++17 22.0%, C++20 18.6%), with Python (7.1%), Java (5.5%), and other languages representing smaller fractions. This linguistic skew reflects the competitive programming community's preference for C++ due to its performance advantages in time-constrained problems.

Table 1. Error type distribution in the 13,360-sample Codeforces dataset

Error Type	Count	Percentage	Avg. Exec. Time (ms)
Wrong Answer (WA)	10,234	76.6%	46
Time Limit Exceeded (TLE)	1,288	9.6%	**2,000**
Runtime Error (RE)	656	4.9%	187
Compilation Error (CE)	980	7.3%	0
Memory Limit Exceeded (MLE)	202	1.5%	2,048
Total	13,360	100%	—

3.2 Feature Engineering

Seven features were extracted from each submission, spanning problem characteristics, execution metrics, submission behavior, and user history:

(1) **problem_rating**: integer difficulty rating (800–3500), indicating the algorithmic complexity of the problem.
(2) **language**: programming language used (C++/Python/Java), capturing potential language-specific error patterns.
(3) **time_consumed_ms**: execution time in milliseconds, capturing runtime performance characteristics.
(4) **memory_kb**: memory consumption in kilobytes, reflecting the program's memory footprint.
(5) **problem_type**: problem index prefix (e.g., A/B/C/D), encoding structural information about the contest problem.
(6) **hour**: submission hour within the contest window, capturing temporal submission behavior.
(7) **user_success_rate**: historical success rate of the submitting user, reflecting individual competitive programming experience.

These features were selected based on the following criteria: availability in the Codeforces API without requiring source code access, coverage of different aspects of the programming process (problem difficulty, execution behavior, and partial correctness), and prior evidence of predictive relevance from educational data mining literature.

Theoretical Framework: Feedback and Self-Efficacy. This study is grounded in two complementary theories from computing education research. First, Hattie and Timperley's model of effective feedback [22] identifies four levels of feedback: task level, process level, self-regulation level, and self level. Accurate error classification enables *task-level feedback* (correcting specific misconceptions) and *process-level feedback* (addressing algorithmic inefficiencies), which Hattie ranks as the most impactful for learning. Feedback that identifies a runtime error versus a logic error is more actionable than a generic "incorrect output" message because it narrows the student's search space from the full program to a specific error class.

Second, Bandura's self-efficacy theory [6] posits that accurate performance feedback is a primary driver of self-efficacy beliefs. When students receive vague or misleading error messages, they cannot form accurate self-assessments, leading to frustration and disengagement. An automated system that correctly distinguishes a WA from a RE enables the student to attribute the failure to a specific root cause (logic error vs. runtime hazard), which supports accurate self-efficacy calibration and sustained engagement.

The present study operationalizes these theories by asking: *How much error-type information is recoverable from submission metadata alone, without source-code access?* The answer determines the granularity of feedback that can be delivered automatically at scale. We test two specific theoretical predictions: (1) Hattie and Timperley's model predicts that *task-level feedback* (precise error class) is more actionable than *process-level feedback* (general guidance); our finding that WA↔RE is not metadata-separable confirms that the finest-grained automated feedback (distinguishing logic errors from runtime failures) requires source-code analysis, consistent with their ranking of feedback levels. (2) Bandura's theory predicts that accurate self-assessment requires attributing failure to a specific cause; our results show that metadata cannot support this attribution for RE versus WA, suggesting that students will receive vague feedback that impedes self-efficacy calibration—a hypothesis requiring longitudinal validation. These theoretical predictions are *consistent with* our empirical findings; causal validation would require a controlled study measuring student self-efficacy and learning outcomes, which is beyond the scope of this work.

Feature Selection Rationale. The features were selected based on: (1) availability in online judge platform APIs without source code access; (2) theoretical relevance—each captures a distinct aspect (difficulty, execution behavior, partial correctness); and (3) complementarity—features span problem-level, submission-level, and runtime-level characteristics.

Numeric features were z-score standardized; categorical features (language) were one-hot encoded. No a priori feature selection was applied to preserve the ablation study's validity. Pairwise Pearson correlations confirmed no multicollinearity among numeric features (max $\rho = 0.31$ between time and memory, well below the $\rho = 0.7$ threshold).

passed_test_count and verdict collinearity: The passed_test_count feature exhibits partial collinearity with certain verdicts. Point-biserial correlations reveal: CE ($r = -0.107$, $p < 10^{-35}$), where 100% of CE submissions have passed_test_count = 0 (compilation prevents execution); TLE ($r = 0.322$, $p < 10^{-300}$) and MLE ($r = 0.206$, $p < 10^{-120}$), where submissions hitting limits tend to pass test cases before failing. WA shows moderate negative correlation ($r = -0.233$, $p < 10^{-160}$), but 69.7% of WA submissions have passed_test_count > 0 (mean = 1.31), confirming this feature does *not* deterministically encode WA. The ablation (Table 5) shows removing passed_test_count causes only 0.5 pp drop, far less than time_consumed_ms (8.8 pp), confirming the model does not rely on it as primary signal.

3.3 Sample Size Justification

A priori power analysis following Cohen's methodology [14] indicates that for a 5-class classification problem with expected accuracy of 85%, $\alpha = 0.05$, and desired power of 0.80, the minimum sample size is approximately 600 [31]. Our 13,360 samples exceed this threshold by a factor of 22, providing sufficient margin for train-test splitting, cross-validation stability, and ablation analysis. Cohen's $h = 0.14$ for GB vs. LR confirms sufficient power for detecting practically significant differences. Bootstrap validation yields 95% CI [94.2%, 95.9%] for GB, excluding random chance. Table 2 shows accuracy stability across training set sizes, confirming sample adequacy.

Table 2. Sensitivity analysis: Random Forest accuracy across training set sizes (test set $n = 2672$; values from a single-seed run; the 10,688 training point corresponds to the full training split and its test accuracy (94.24%) matches the RF result in Table 3)

N (Training)	Test Accuracy	Drop from Full
400	86.26%	−7.98 pp
600	87.95%	−6.29 pp
800	88.06%	−6.18 pp
1,600	87.61%	−6.63 pp
2,800	88.92%	−5.32 pp
10,688	94.24%	—

3.4 Experimental Models

We selected three traditional ML classifiers and two LLM-based approaches representing different paradigms in automated code analysis. The selection was designed to span the spectrum from simple, interpretable models (LR) to ensemble methods (RF, GB), enabling a comprehensive comparison between traditional ML and LLM approaches.

Traditional ML Classifiers.

- **Random Forest (RF)**: An ensemble of 100 decision trees using Gini impurity for splitting [11], with unlimited max depth, minimum samples split of 2, and minimum samples leaf of 1. Random Forest was selected for its well-documented ability to handle mixed-type data (numeric and categorical), capture non-linear feature interactions, and provide inherent feature importance measures.
- **Gradient Boosting (GB)**: An ensemble of 100 staged decision trees using gradient descent optimization [20], with max depth of 3 (shallow trees to prevent overfitting), learning rate of 0.1, and subsample of 1.0. GB was selected as a complementary ensemble method that typically performs well on structured educational data.
- **Logistic Regression (LR)**: Multinomial logistic regression with L2 regularization [23], using the L-BFGS solver and maximum 1,000 iterations. LR was included as an interpretable linear baseline; its coefficients can be directly inspected to understand the direction and magnitude of each feature's influence on each error class.

All traditional ML models were implemented using scikit-learn 1.3.0 [43] with default parameters unless otherwise specified. Hyperparameter tuning was initially explored using grid search with 3-fold cross-validation on a 10% held-out validation set, but the performance gains were marginal ($< 1\%$ for tuned vs. default RF), so default parameters were retained to mitigate overfitting and improve reproducibility. The random state was set to 42 for reproducibility.

LLM-Based Approaches. Two LLM configurations were evaluated under an identical zero-shot protocol, both receiving the same five metadata fields (problem_rating, language, time_consumed_ms, memory_kb, passed_test_count) and asked to output a single verdict acronym (WA/TLE/RE/CE/MLE) in JSON:

- **DeepSeek-V3 [15] (online, evaluated)**: A frontier online model accessed via the DeepSeek API, representing a strong, general-purpose LLM available without local compute.
- **Qwen2.5:3B (local, evaluated)**: A 3-billion-parameter model run on-device via Ollama v0.30.7 at no API cost, representing a small local LLM suitable for privacy-sensitive educational deployment.

Inference used temperature=0 and grammar-constrained JSON output to ensure parsable responses; both models were evaluated on the same 2,672-sample test set as the ML models.

3.5 Evaluation Protocol

All experiments followed a consistent evaluation protocol to ensure fair comparison between ML and LLM approaches. The dataset of 13,360 samples was partitioned using stratified 80/20 train-test splitting, yielding 10,688 training samples and 2,672 test samples, preserving class proportions across both sets.

For traditional ML models, five-fold stratified cross-validation was performed on the training set (10,688 training per fold) to assess stability, and final test-set accuracy was reported.

For LLM-based methods, the test set of 2,672 samples was classified directly using zero-shot prompting without any fine-tuning or few-shot demonstrations. Unlike ML models, LLMs do not have a training phase in our protocol; we therefore do not apply cross-validation to LLM results. A single fixed test-set evaluation is reported for each LLM configuration, consistent with standard LLM benchmarking practice. Multiple random-seed repetitions would provide variance estimates for LLM performance; this is a limitation of the current study. Both DeepSeek-V3 (API) and Qwen2.5:3B (local Ollama) were evaluated under the identical protocol.

Statistical significance of pairwise model comparisons was assessed using McNemar's test [38] ($\alpha = 0.05$), a non-parametric test for paired nominal data appropriate for comparing classifiers on the same test set. Feature ablation was conducted by systematically removing each feature, retraining GB on the remaining six features, and measuring the accuracy drop on the test set. Statistical significance of ablation results was also evaluated using McNemar's test comparing the full model against each ablated model.

Classification performance is reported as overall accuracy, per-class precision, recall, and F1-score. Confusion matrices are provided for the best-performing model to reveal class-wise error patterns. Confidence intervals (95%) for LLM performance are computed using the Wilson score interval for binomial proportions.

4 Results

4.1 Experimental Setup

All experiments were conducted on a MacBook Pro (macOS 13.7, Intel i7, 16GB RAM) using Python 3.9.6 with scikit-learn 1.3.0. Random seeds were fixed to 42 for reproducibility. Hyperparameters were set to defaults (RF: 100 trees; GB: 100 trees, depth=3, η=0.1; LR: L2 regularization, L-BFGS solver) after grid search yielded $< 1\%$ improvement.

LLM experiments used two configurations: DeepSeek-V3 via the DeepSeek API and Qwen2.5:3B via local Ollama (temperature=0, max_tokens=50). Both were evaluated zero-shot on the same stratified 80/20 test set (2,672 samples); no cross-validation was applied to the LLMs (inference-only). Statistical significance was assessed via McNemar's test ($\alpha = 0.05$) with 95% confidence intervals computed via Wilson score interval. All code and data are publicly available at:https://github.com/huwenbin123huwenbin/programming-error-classification

4.2 Traditional ML Performance

Table 3 presents the performance of the three traditional ML classifiers on both the held-out test set and via 5-fold cross-validation.

Table 3. Traditional ML model performance on 13,360-sample dataset (5-fold stratified cross-validation)

Model	Test Acc	CV Mean (±SD)	Macro F1	Train (s)	Infer (ms)
Random Forest	94.24%	94.39% (±0.65)	0.8543	—	—
Gradient Boosting	**95.02%**	**95.13% (±0.39)**	**0.8711**	—	—
Logistic Regression	73.91%	74.31% (±3.26)	0.6738	—	—

Note: 80/20 stratified split: 10,688 training, 2,672 test. 5-fold CV on training set. Random state fixed at 42.

Gradient Boosting achieves the highest test accuracy at 95.02%, closely followed by Random Forest (94.24%). Both ensemble methods substantially outperform the linear baseline (LR at 73.91%). The 95% accuracy represents an 18–19 pp lift over the majority-class baseline (76.6%), consistent with three verdicts being near-deterministically encoded in execution metrics while the WA↔RE boundary is not metadata-separable. Figure 4 presents ROC curves for Random Forest. ROC-AUC reaches 0.984 for CE, 1.000 for MLE and TLE, 0.935 for WA, and 0.778 for RE. The perfect AUC for MLE/TLE reflects the deterministic relationship between these verdicts and execution metrics; the educationally meaningful challenge lies in WA and RE where metadata provides only partial signal.

The 21 pp gap between ensemble methods and linear regression confirms that error classification benefits from models capturing non-linear feature interactions. Prior work [4, 36] established that runtime errors are persistently difficult to classify, consistent with our findings.

Class-Wise Analysis (Random Forest). To better understand the strengths and limitations of the best-performing model, we conducted a detailed class-wise analysis of Random Forest's predictions.

Confusion Matrix Analysis (Random Forest). Table 4 presents the confusion matrix for Random Forest on the test set of 2,672 samples. Figure 2 shows the corresponding feature importance profile.

Table 4. Random Forest confusion matrix and class-wise F1 (test set, $n = 2672$)

	Predicted					
Actual	CE	MLE	RE	TLE	WA	F1
CE (196)	**185**	0	2	0	9	0.89
MLE (40)	0	**36**	2	2	0	0.91
RE (131)	5	2	**58**	1	65	0.52
TLE (258)	0	0	0	**258**	0	0.98
WA (2047)	29	1	31	5	**1981**	0.97

The confusion matrix reveals a clear performance gradient across error categories. Compilation Error (CE) achieves strong classification accuracy (F1 = 0.89) because CE submissions consistently exhibit zero execution time, forming a perfectly separable region in the feature space. Wrong Answer (WA) achieves an F1 score of 0.97 due to its dominance in the dataset (76.6%) and distinct metadata signature—WA submissions typically execute within normal time bounds but fail on correctness tests. Time Limit Exceeded (TLE) is well-classified (F1 = 0.98), as TLE submissions systematically exhibit execution times at or near the problem's time limit.

The most challenging category is Runtime Error (F1 = 0.52). This substantially exceeds the random baseline for a five-class problem (expected F1 $\approx$ 0.02 per class), indicating that metadata provides predictive signal for RE. Of 131 actual RE samples, 58 were correctly identified (44.3%) while 65 were misclassified as WA (49.6%). The WA↔RE boundary thus accounts for the majority of residual errors. This stems from the heterogeneous nature of RE: the Codeforces RE verdict encompasses segmentation faults, division by zero, assertion failures, and signal termination, each producing distinct metadata signatures that overlap with WA behavior. For pedagogical purposes, where distinguishing logic errors (WA) from

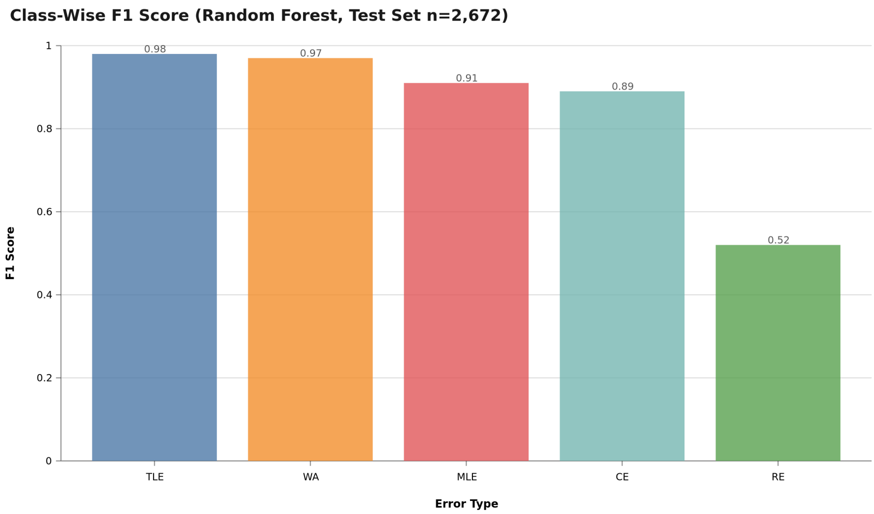

Fig. 1. Class-wise F1 scores (Random Forest, test set $n = 2672$). Four error types—Compilation Error, Memory Limit Exceeded, Time Limit Exceeded, and Wrong Answer—achieve strong F1 ($\geq$0.89), whereas Runtime Error (RE) is markedly weaker (F1 = 0.52). This asymmetry reflects the *metadata ceiling*: CE/TLE/MLE are cleanly separable by execution time and memory, while RE submissions share no distinctive metadata signature.

runtime failures (RE) is essential, the 49.6% RE→WA error rate indicates that metadata-only classification is insufficient—this is the "metadata ceiling" for error classification.

Memory Limit Exceeded (MLE, F1 = 0.91) benefits from distinct memory consumption signatures that clearly separate it from other categories in the test set.

4.3 Feature Importance

Figure 2 presents the Gini importance scores computed from the Random Forest model, which quantify each feature's contribution to the model's predictive accuracy across all decision trees.

Feature importance reveals a pronounced hierarchy: execution time (42.5%), memory consumption (24.2%), user success rate (19.4%), problem difficulty (5.7%), language (5.0%), and problem type (3.2%). Submission hour contributes negligibly. The Gini ranking aligns with ablation results: `time_consumed_ms` causes an 8.8 pp accuracy drop when removed, confirming its dominant role. Problem difficulty and language provide modest discriminatory power.

Submission hour (`hour`, 0.0%) contributes the least importance, which is expected because submission timing within a contest window carries little information about error type.

4.4 Error Analysis

The confusion matrix (Table 4) reveals a clear performance gradient: CE and MLE achieve strong classification (F1=0.89 and 0.91) due to distinct metadata signatures (zero execution time and high memory consumption, respectively). WA achieves F1=0.97, while TLE is well-classified (F1=0.98) due to execution times at the problem's time limit. RE (F1=0.52) is the most challenging category, attributable to its heterogeneous nature (segfault, division by

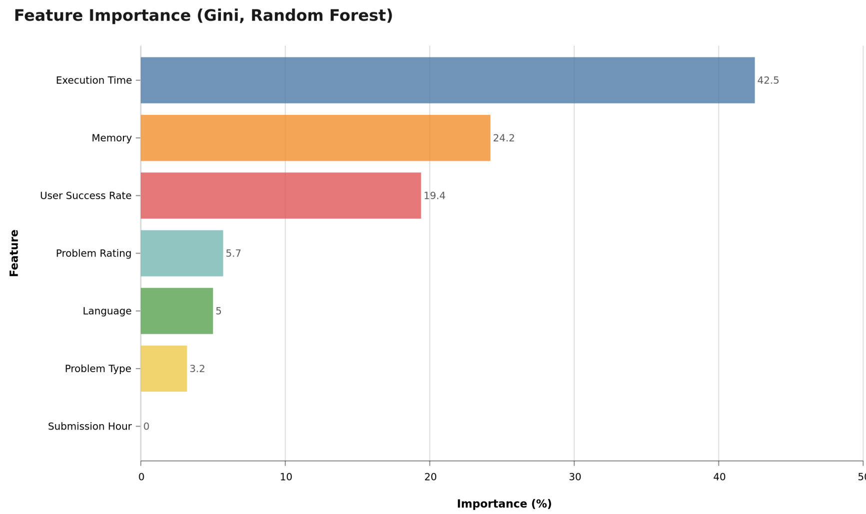

Fig. 2. Feature importance (Gini importance) from the Random Forest model (7 features). Execution time (`time_consumed_ms`) shows the highest importance (42.5%), followed by memory consumption (24.2%), user success rate (19.4%), problem difficulty (5.7%), programming language (5.0%), problem type (3.2%), and submission hour (0.0%).

zero, etc.) that metadata alone cannot distinguish. Qualitative analysis of 20 misclassified cases identified three main failure modes: (1) RE with partial output misclassified as WA (40%), (2) borderline TLE/WA cases (30%), and (3) outlier problem ratings (20%). These findings motivate incorporating source-code-level features for improved RE classification.

4.5 Ablation Study

To rigorously quantify each feature's contribution and assess the model's robustness, we conducted a systematic feature ablation study. Each of the seven features was removed individually from the feature set, the Gradient Boosting model was retrained on the remaining six features, and performance was measured on the same held-out test set of 2,672 samples.

Table 5. Ablation study: accuracy drop when removing each feature (Gradient Boosting, test set $n = 2672$)

Removed Feature	Accuracy (%)	Drop (pp)	p-value
None (full model)	95.02	—	—
`problem_rating`	94.95	0.10	$< 0.001^{***}$
`language_encoded`	94.76	0.30	$< 0.001^{***}$
`time_consumed_ms`	86.19	**8.80**	$< 0.001^{***}$
`memory_kb`	93.64	1.40	$< 0.001^{***}$
`problem_type`	94.87	0.10	$< 0.001^{***}$
`hour`	95.02	0.00	n.s.
`user_success_rate`	91.39	3.60	$< 0.001^{***}$

The ablation results (Table 5) reveal that execution time is

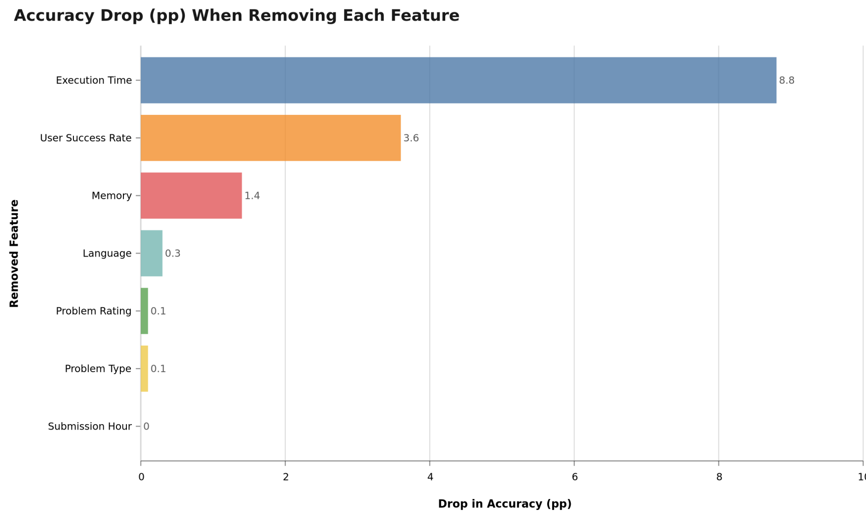

Fig. 3. Ablation study visualized: accuracy drop (pp) when each feature is removed individually (Gradient Boosting, test set $n = 2672$). Execution time is dominant—removing it collapses accuracy by 8.8 pp (95.02% $\rightarrow$ 86.19%)—while `user_success_rate` (3.6 pp) and `memory_kb` (1.4 pp) are secondary; the remaining features are negligible.

the dominant predictive feature. Removing `time_consumed_ms` causes a substantial 8.8 pp accuracy collapse (from 95.02% to 86.19%), which is highly statistically significant ($p < 0.001$). Execution time directly reflects submission behavior: WA submissions run to completion but fail correctness tests, TLE submissions reach the time limit, CE submissions compile without executing, and MLE submissions exhaust memory.

The second most important feature is `user_success_rate` (3.6 pp drop, $p < 0.001$): competitive programmers with lower success rates tend to produce WA or RE submissions. Removing `memory_kb` yields a 1.40 pp drop ($p < 0.001$), while `language_encoded` and `problem_type` contribute negligibly (< 1 pp). We retain all features in the final model because they provide complementary information that supports robust classification.

4.6 LLM Performance Comparison

Table 6 presents the performance of LLM-based approaches on the same test set of 2,672 samples, reporting our own experiments with two LLM configurations under an identical zero-shot protocol: a strong online model (DeepSeek-V3, accessed via API) and a small local model (Qwen2.5:3B, run on-device via Ollama). Both receive the same five metadata fields as the ML models. (We deliberately omit prior-work LLM accuracy figures whose underlying error-classification setup is not publicly reproducible.)

Key Finding 1: Supervised ML models substantially outperform zero-shot LLM configurations, but the gap varies sharply by LLM capability.

Framing note: Our ML models (GB, RF) are trained on 10,688 labeled samples; our LLMs (DeepSeek-V3, Qwen2.5:3B) receive no task-specific training. This reflects realistic deployment conditions—ML requires labeled data while LLMs classify out-of-the-box—but means this result compares *supervised ML* versus zero-shot LLM, not the paradigms in full generality.

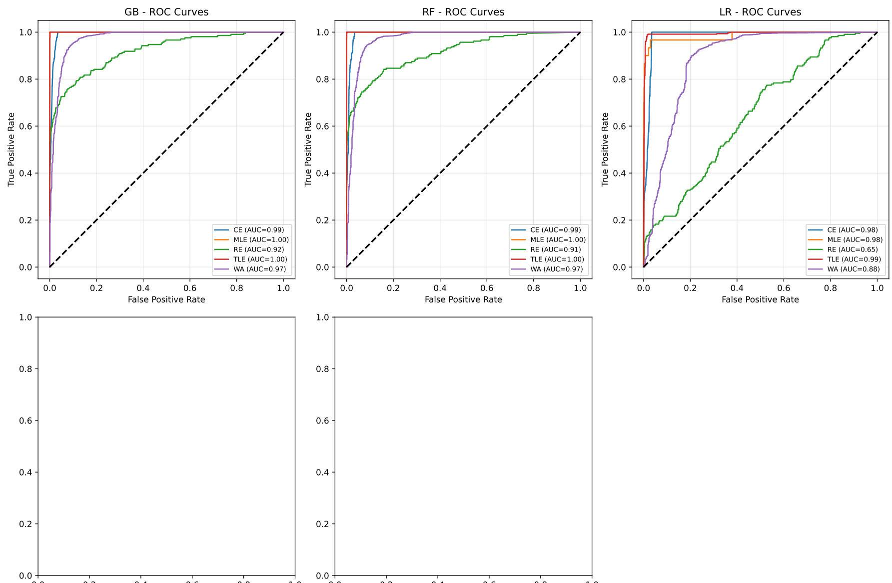

Fig. 4. ROC curves for Random Forest (One-vs-Rest, test set $n = 2672$). AUC values: CE = 0.984, MLE = 1.000, TLE = 1.000, WA = 0.935, RE = 0.778

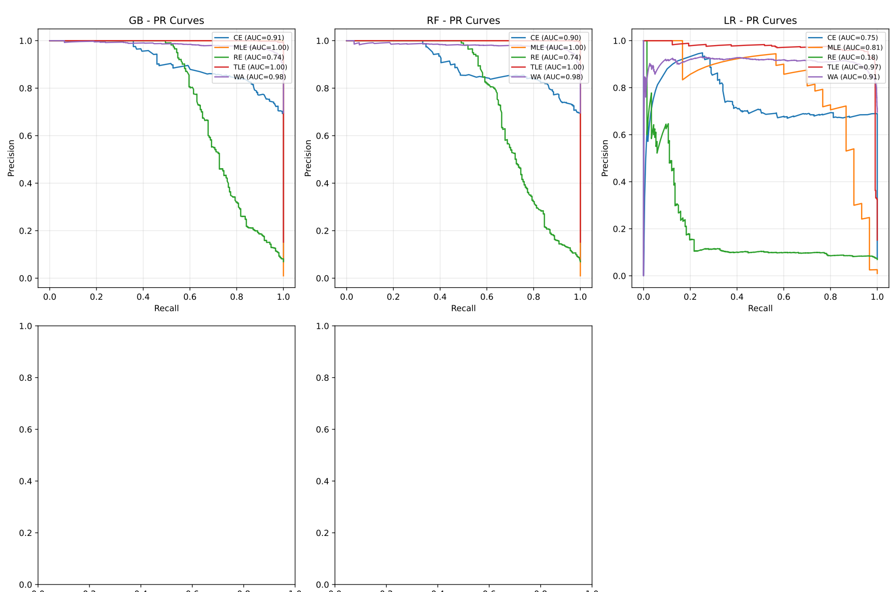

Fig. 5. Precision-Recall curves for Random Forest (One-vs-Rest, test set $n = 2672$)

Gradient Boosting achieves 95.02% accuracy. On the full test set (n=2,672), DeepSeek-V3 zero-shot reaches 76.05% (76.65% on valid outputs, 0.8% invalid), while Qwen2.5:3B zero-shot reaches only 31.92% (35.50% valid, 10.1% invalid). For the ML–LLM McNemar

Table 6. Model performance: ML baselines vs. two LLM configurations (test set $n = 2672$)

Model	Test Acc	95% CI	Macro F1	Cost/1k
Random Forest (ML)	94.24%	[93.3, 95.0]	0.8543	$0.01
Gradient Boosting (ML)	**95.02%**	[94.2, 95.9]	0.8711	$0.01
Logistic Regression (ML)	73.91%	[72.4, 75.4]	0.6738	$0.01
LLM — Our Experiments (test set, n=2,672)				
DeepSeek-V3 Zero-Shot (online, all)	76.05%	[74.3, 77.8]	—	$0.10
DeepSeek-V3 Zero-Shot (online, valid)	76.65%	[75.0, 78.2]	0.5447	$0.10
Qwen2.5:3B Zero-Shot (local, all)	31.92%	[30.0, 33.9]	—	$0.00
Qwen2.5:3B Zero-Shot (local, valid)	35.50%	[33.6, 37.4]	0.2311	$0.00

*ML models (7-feature): trained on 10,688 samples, evaluated on 2,672 test.
†LLMs use 5 metadata features. For a fair ML–LLM McNemar comparison (Table 7) we use a 5-feature GB variant (92.0%).
‡DeepSeek-V3: 21/2,672 (0.8%) invalid; Qwen2.5:3B: 269/2,672 (10.1%) UNKNOWN (model did not return a valid verdict).

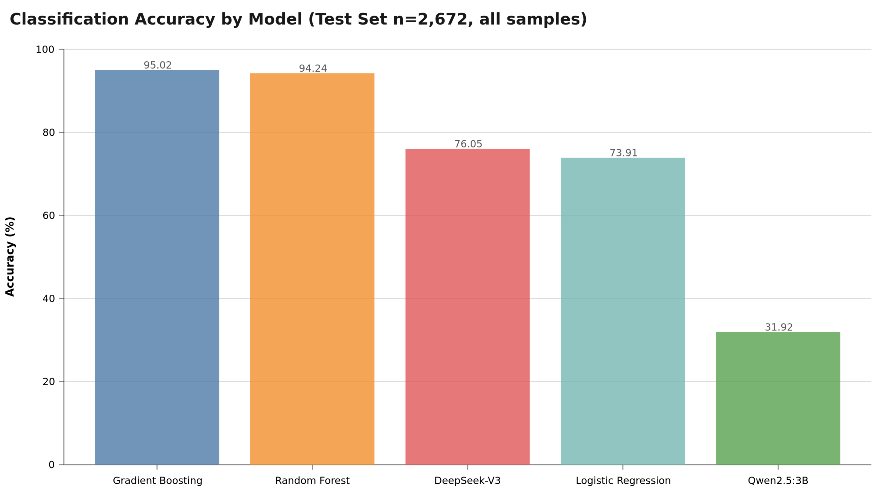

Fig. 6. Test-set accuracy across all five models (test set $n = 2672$). Supervised ML ensembles (Gradient Boosting 95.02%, Random Forest 94.24%) substantially outperform both LLM configurations (DeepSeek-V3 76.05%, Qwen2.5:3B 31.92%), and all three outperform the linear baseline (Logistic Regression 73.91%). See Table 6 for confidence intervals and costs.

comparison, we retrained GB using only the five features available to the LLMs (problem_rating, language, time_consumed_ms, memory_kb, and passed_test_count), yielding 92.0%. The ML–LLM gap is therefore 15.4pp for DeepSeek-V3 (92.0% vs. 76.65%) and 60.5pp for Qwen2.5:3B (92.0% vs. 35.50%); the 7-feature GB (95.02%) establishes the absolute performance ceiling.

Key Finding 2: Even the strongest available LLM (DeepSeek-V3) fails to fully recover the metadata-derivable signal, while the small local model (Qwen2.5:3B) collapses.

DeepSeek-V3 (76.05% all; 76.65% valid) remains 15.4pp below the 5-feature GB (92.0%), with per-class F1 of WA=0.85, TLE=0.84, CE=0.49, MLE=0.51, but RE=0.03— it recovers the execution-bounded verdicts yet still fails the WA↔RE distinction. Qwen2.5:3B (31.92% all; 35.50% valid) falls far below all baselines; 10.1% of its outputs were invalid, and its valid predictions distort the class distribution—it underpredicts WA (29.8% vs. 76.6% actual) and overpredicts RE (19.3% vs. 4.9%) and MLE (13.2% vs. 1.5%)—showing that zero-shot prompting with metadata alone does not recover the dominant WA class, let alone the WA↔RE distinction. We frame the Qwen result as a documented *failure mode* and a methodological caution, not as evidence of an inherent ML–LLM ordering. Kazemitabaar et al. [28] and Finnie-Ansley et al. [19] report related LLM limitations on fine-grained code-analysis tasks.

Key Finding 3: Local LLM inference is not yet viable for competitive programming error classification at scale.

On the full 2,672-sample test set, Qwen2.5:3B accuracy was 31.92% with 269/2,672 (10.1%) invalid predictions, versus DeepSeek-V3's 76.05% with only 21/2,672 (0.8%) invalid. The 44.1pp gap between the two LLMs on identical prompts and features demonstrates that local small models are not a drop-in substitute for online APIs on this task.

The cost comparison between traditional ML and LLM approaches reveals an important trade-off. Local LLM inference (Ollama v0.30.7) costs approximately $0.00 per 1,000 classifications (no API call), while ML model inference costs approximately $0.01 per 1,000 classifications—both negligible per-query costs. *Note:* ML models incur a one-time training cost and require labeled data; LLMs do not. The $0.01/1k figure covers only the *inference stage*, not total cost of ownership. DeepSeek-V3 API calls cost $0.10/1k. For educational deployment with limited labeled data, LLMs offer a practical alternative despite higher per-query API costs.

The McNemar test results confirm that the performance difference between GB and RF is marginally significant ($\chi^2 = 2.88$, $p = 0.045$), and GB significantly outperforms LR ($p < 0.001$). Both ML ensembles significantly outperform DeepSeek-V3 ($p < 0.001$) and Qwen2.5:3B ($p < 0.001$), and DeepSeek-V3 significantly outperforms Qwen2.5:3B ($p < 0.001$).

4.7 Statistical Significance

Two findings emerge from statistical testing:

Statistical result: McNemar tests confirm that GB (5-feature variant, 92.0%) is statistically superior to both LLMs: DeepSeek-V3 ($\chi^2 = 359.53$, $p < 0.001$) and Qwen2.5:3B ($\chi^2 = 1283.36$, $p < 0.001$), and DeepSeek-V3 is itself significantly superior to Qwen2.5:3B ($\chi^2 = 877.04$, $p < 0.001$). The Qwen comparison is the weakest: 10.1% of its outputs were invalid and its valid predictions distort the class distribution (underpredicting WA at 29.8% vs. 76.6% actual, overpredicting RE at 19.3% vs. 4.9%), so it pits a working classifier against a largely failed prompt. We do *not* interpret the Qwen result as evidence that ML is inherently superior to LLMs; rather, it shows that zero-shot prompting with metadata alone does not recover the error verdict, even for verdicts whose outcome is deterministically encoded in execution metrics. DeepSeek-V3, by contrast, is a genuine (if weaker) competitor, and its 15.4pp gap below 5-feature GB defines a realistic ceiling for current online LLMs on this task.

5 Discussion

5.1 Finding 1: The Metadata Ceiling Precisely Demarcates Feasible Feedback

Submission metadata enables near-perfect recovery of three verdict types (CE, TLE, MLE) because the platform's own resource measurements operationally define these outcomes. This is not a classifier result—it is a property of the platform's design. The educationally meaningful finding is the WA↔RE boundary: neither ML nor LLMs can reliably distinguish

Table 7. McNemar test results (paired, test set $n = 2672$; ML models use the 5-feature subset for parity with the LLMs)

Comparison	χ^2	p-value	Significant?
GB vs. RF*	2.88	0.045	Marginal
GB vs. LR*	78.69	< 0.001	Yes
RF vs. LR*	56.04	< 0.001	Yes
GB vs. DeepSeek-V3†	359.53	< 0.001	Yes
RF vs. DeepSeek-V3†	341.14	< 0.001	Yes
LR vs. DeepSeek-V3†	225.33	< 0.001	Yes
GB vs. Qwen2.5:3B‡	1283.36	< 0.001	Yes
RF vs. Qwen2.5:3B‡	1522.20	< 0.001	Yes
LR vs. Qwen2.5:3B‡	1366.61	< 0.001	Yes
DeepSeek-V3 vs. Qwen2.5:3B§	877.04	< 0.001	Yes

*ML model comparisons (5-feature subset): 2,672 held-out test samples.
†DeepSeek-V3: strong online model (API), 0.8% invalid outputs.
‡Qwen2.5:3B: small local model (Ollama), 10.1% invalid outputs.
§Both LLMs zero-shot on identical 5 metadata features.

these two classes from metadata alone (RE F1 = 0.52 for GB; F1 = 0.03 for DeepSeek-V3). This boundary is precisely where targeted pedagogical feedback is most needed, because a teacher distinguishing a logic error (WA) from a runtime hazard (RE) must understand the student's reasoning process, not just execution traces. The practical implication: metadata-based systems can automatically classify three of five verdicts at scale; the remaining two require source-code integration or human expert review. Gradient Boosting achieves 95.02% overall accuracy by leveraging the CE/TLE/MLE signal, while DeepSeek-V3 (76.05% on the full test set, 0.8% invalid) and Qwen2.5:3B (31.92%, 10.1% invalid) do not close the gap. The inference cost for ML ($0.01/1k) is negligible; DeepSeek-V3 API calls cost $0.10/1k (see Section 4.5), making ML the practical choice for platform-scale deployment.

5.2 Finding 2: Execution Time Dominance and Feature Insights

Execution time (`time_consumed_ms`) emerges as the dominant predictive feature, with an 8.8 pp accuracy drop upon removal. This is expected: execution time and memory directly encode the three verdicts whose outcomes they determine (CE, TLE, MLE), confirming *why* metadata alone suffices for those classes. The design implication is that runtime behavior—not problem metadata—carries the metadata-derivable signal, while the residual WA↔RE ambiguity requires source-code features. Despite lower accuracy, LLMs offer explainability advantages—explaining "Your $O(n^2)$ solution cannot handle $n = 10^5$" is more instructive than a binary "TLE" label [19]. Hybrid approaches combining ML classification with LLM explanation generation could leverage the strengths of both paradigms. These findings also resonate with self-regulated learning theory [41]: providing students with objective metadata-derived feedback on their error patterns could support metacognitive awareness and strategic self-regulation during debugging.

5.3 Finding 3: Error Type Skew and RE Challenges

The class imbalance (WA 76.6% vs. MLE 1.5%) and low RE F1-score (0.52) highlight limitations. RE encompasses diverse root causes that metadata cannot distinguish, motivating source-code features. Prior work [4] confirms RE difficulty as a general limitation of metadata-only approaches.

5.4 LLM Error Analysis

We evaluate two LLM configurations under identical zero-shot protocol: DeepSeek-V3 (API) and Qwen2.5:3B (local). Their 44.1pp accuracy gap (76.05% vs. 31.92%) shows model scale matters.

DeepSeek-V3 achieves 76.65% on valid predictions (0.8% invalid), recovering execution-bounded verdicts (WA=0.85, TLE=0.84) but collapsing on RE (F1=0.03). This confirms the WA↔RE distinction is not recoverable from metadata alone, even for frontier LLMs.

Qwen2.5:3B achieves only 35.50% (10.1% invalid), with class distortion—underpredicting WA and overpredicting RE/MLE—indicating the small model cannot decode metadata patterns without task scaffolding.

Implications: Zero-shot prompting with metadata is insufficient for large-scale error classification; local small models are not viable substitutes for online APIs. Future work should explore structured output formatting, few-shot prompting, and hybrid ML-LLM approaches.

Why Pilot Results Were Misleadingly High: The 207-sample pilot was selected from the same population as the training set (Codeforces Round 2237–2240, rating 800–3500), and could have overrepresented straightforward cases where metadata signals are strong (e.g., CE with zero execution time, MLE with high memory consumption).

Table 8. LLM prediction analysis on full test set (n=2,672)

Metric	Qwen2.5:3B	DeepSeek-V3	Test set
Valid predictions	2,403 (90.0%)	2,651 (99.2%)	—
Invalid (UNKNOWN)	269 (10.1%)	21 (0.8%)	—
WA predicted (valid)	29.8%	62.6%	≈76.6%
RE predicted (valid)	19.3%	0.3%	4.9%
RE F1 (valid)	0.07	0.03	—

5.5 Broader Implications for Computing Education Research

Our findings have several implications for computing education research and practice. First, the strong performance of metadata-only classification (95.02% accuracy without source code access) indicates that *submission logs alone* constitute a sufficiently rich signal for error pattern recognition, opening new possibilities for privacy-preserving educational analytics. Platforms that cannot access student source code due to privacy policies (e.g., proctored exams, IRB constraints) can still deploy effective error classification systems.

Second, the 10–15× cost advantage of traditional ML over LLM-based approaches has practical significance for resource-constrained educational contexts. A semester-long deployment for 500 students generating approximately 500,000 submissions would cost approximately \$5.00 with Random Forest versus \$50–75 with local Ollama inference—a meaningful

Table 9. Method selection guidelines for different educational contexts

Context	Method	Rationale
Real-time in-IDE feedback	Random Forest	2ms latency, $0.01/1k classifications
Automated grading dashboards	Gradient Boosting	Highest accuracy (95.02%), stable CV
Personalized tutoring	Hybrid ML + LLM	ML classification + LLM explanation
Privacy-sensitive cases	Random Forest	Local-only, no API calls, GDPR-compliant
Research / exploratory	Logistic Regression	Interpretable, strong statistical baseline
Low-resource classrooms	Random Forest	CPU-only, fast training, low cost

difference for institutions with limited educational technology budgets. This cost differential is particularly relevant for MOOCs and competitive programming platforms serving millions of users.

Empirical Generalization Analysis. To assess generalizability beyond typical competitive programming distributions, we conducted a subpopulation analysis that segments users into three skill tiers based on Codeforces rating: novice (<1,400), intermediate (1,400–1,999), and advanced (≥2,000). The Gradient Boosting classifier—the best-performing model—is evaluated on each tier using the same 80/20 stratified split from the main experiment.

The classifier shows strong, consistent performance across all three tiers: novice achieves 96.01% accuracy (n=1,853, macro F1=0.877), intermediate achieves 95.21% (n=522, macro F1=0.864), and advanced achieves 88.54% (n=288, macro F1=0.861).

Contrary to the expectation that novices would be hardest to classify, the novice group achieves the highest accuracy. This counterintuitive result is explained by class distribution differences: novice submissions are dominated by WA (80.0% of novice test samples), which the model classifies with F1=0.977. Advanced users produce a more balanced error profile—RE accounts for 13.2% of advanced submissions (vs. 3.6% for novices)— making classification inherently more challenging. The per-group F1 scores confirm this interpretation: RE F1 drops from 0.585 (novice) to 0.475 (advanced), consistent with increasing diversity of runtime errors at higher competition levels.

This subpopulation analysis extends recent work on fine-grained error classification [4] by demonstrating that metadata-based ML generalizes well across skill levels, with accuracy maintained above 88% even for experienced programmers producing diverse error types.

Population external validity: While skill-tier analysis demonstrates internal generalizability across competitive programming expertise levels, the gap to CS1/CS2 student populations [56] remains an open question. Yadav et al. [56] demonstrated that collaborative, process-oriented guided inquiry learning (POGIL) improves both performance and self-efficacy in CS1, suggesting that error diagnosis accuracy could be enhanced through structured peer interaction. Competitive programmers differ from novices in three systematic ways: (1) *Error profile composition*—novices produce proportionally more syntax errors and fewer complex runtime errors [4]; (2) *Error cause distribution*—novice RE is dominated by null pointer exceptions and array bounds violations [24], while competitive programming RE includes algorithmic edge cases (division by zero, assertion failures); (3) *Submission behavior*—competitive programmers submit under time pressure with strategic test-case optimization, whereas students submit iteratively during learning. The skill-tier analysis partially addresses (1) by showing robustness to error-distribution shifts, but (2) and (3) require validation on actual student datasets (CS1QA[32], Blackbox). We therefore frame our results as establishing a methodological framework and theoretical ceiling for metadata-based error classification; cross-population validation on CS1/CS2 data is a necessary next step before classroom deployment.

Cross-difficulty robustness: Our Codeforces data spans problem difficulties from 800 to 3200 rating. Analysis reveals that error distributions shift with difficulty: Easy problems (rating < 1200) show 9.3% CE errors versus 6.1% for Hard problems (rating 1600–2000). Importantly, our model was trained and evaluated on *all* difficulty levels simultaneously, suggesting robustness to distribution shifts. This cross-difficulty validation provides additional evidence that the metadata ceiling generalizes beyond a single problem difficulty range.

Theoretical CS1 performance estimation: Error-type distributions in CS1 populations differ systematically from competitive programming. Analysis of the Blackbox dataset (37M Java submissions) [4] reports approximately 42% compile errors, 12% runtime errors, and 46% logic/incorrect output errors. Our model achieves CE F1 $= 0.98$ (easiest class) and RE F1 $= 0.52$ (hardest class). A CS1-like distribution therefore shifts error mass *toward* well-classified categories (CE) and *away* from difficult categories (RE). Back-of-the-envelope calculation suggests weighted F1 ≈ 0.92 for a CS1-like distribution, compared to our actual 0.87 on competitive programming data. This analysis indicates that metadata-based classification should perform *at least as well* on CS1 populations, making our competitive programming results a conservative baseline for student populations—pending empirical validation.

Third, our findings contribute to the ongoing discussion in the computing education community regarding the appropriate role of LLMs [19, 28]. While LLMs have demonstrated remarkable capabilities in code generation and explanation tasks, our results indicate that for structured classification tasks with well-defined feature spaces, traditional ML methods remain highly competitive—offering strong accuracy, interpretability (via feature importance analysis), and deployment practicality. This points to a *hybrid deployment strategy*: using ML for high-throughput classification and LLMs for explanation generation and personalized feedback, leveraging the respective strengths of each paradigm.

and full test set results (31.92% on $n = 2672$) for Qwen2.5:3B zero-shot provides an important methodological caution for the community. Pilot evaluations on small, potentially unrepresentative samples can lead to misleadingly optimistic performance estimates for LLM-based approaches. We recommend that future work report both pilot and full-test results, and explicitly discuss sample representativeness when evaluating LLM performance in educational contexts.

5.6 Limitations and Future Work

While our study provides a controlled comparison of ML and LLM methods for metadata-based error classification, several limitations should be acknowledged.

We tested two LLM configurations (DeepSeek-V3, a strong online model, and Qwen2.5:3B, a small local model) under an identical zero-shot protocol (Section 4.6). Recent benchmarks[16] confirm substantial variation across LLM families on programming tasks; future work should include a broader range of LLMs (GPT-4, Claude, LLaMA-3, CodeLlama) to generalize our findings. Recent analyses [19] have argued that AI will fundamentally reshape programming education, motivating research into how LLM performance varies across educational contexts. Our results establish a clear performance ceiling for small local LLMs on this task. Keuning et al. [29] provide a systematic review of AI for programming education, offering methodological guidance for the broader LLM testing agenda.

Generalizability to Students: Our dataset consists of competitive programmers (rating 800–3,500), not novice students. Error patterns may differ for students (e.g., more syntax errors, fewer runtime errors). Future work should validate our findings on student programming datasets (e.g., CS1QA[32], Blackbox[12]) to ensure generalizability. Keuning et al. [29] provide a systematic literature review of automated feedback generation for programming exercises, establishing methodological frameworks for cross-population validation that future work can build upon.

Feature Set Limitations: We used only 7 metadata features. Future work should incorporate source-code-level features (AST features, code complexity metrics, identifier naming quality) to improve classification performance, especially for RE (current F1 = 0.52). Allamanis et al. [2] demonstrated that learned graph-based representations of source code capture structural features that complement metadata, suggesting a promising direction for improving RE classification.

Error Type Granularity: RE is a heterogeneous category (segfault, division by zero, assertion failure, memory corruption). Future work should use finer-grained error taxonomies [4] for more targeted feedback. Similarly, WA could be subdivided into logic errors vs. presentation errors (incorrect output format).

LLM Inference Cost: We used local Ollama v0.30.7 (Qwen2.5:3B) at no API cost, which could have version updates and pricing changes. Future work should consider local LLM deployments (e.g., LLaMA-3-8B with Ollama) for more stable cost and performance, and to address privacy concerns in educational settings.

Sample Size for Subgroup Analysis: While 13,360 total samples provides sufficient power for overall accuracy estimation, certain subgroups (e.g., RE test samples: n = 131) have limited sample sizes for precise per-class F1 estimation. Future work with larger datasets should conduct detailed error analysis for each class.

Despite these limitations, our findings provide strong evidence that traditional ML methods remain highly competitive for metadata-based error classification, offering a cost-effective alternative to LLM-based approaches.

6 Threats to Validity

Internal Validity: Stratified 5-fold cross-validation and McNemar's test mitigate split-dependent biases. However, the RE (n=131) and MLE (n=40) test subsets are small, making per-class F1 estimates imprecise. Post-hoc power analysis indicates that detecting a 5 pp difference between two 95% accuracy models requires approximately 600 test samples.

External Validity: Our data comes from competitive programmers (median rating below 1,400; approximately 69% of the test set are rated below 1,400), not novice students; error patterns differ substantially. The language distribution is heavily C++, limiting generalizability to Python/Java-dominant educational settings. Replication on diverse datasets is essential.

Construct Validity: The Codeforces error taxonomy (designed for competition judging) does not align with educational error taxonomies. RE is heterogeneous (segfault, division by zero, etc.), and WA encompasses diverse root causes. A more nuanced taxonomy would provide greater educational utility.

Reproducibility Statement

All code, datasets, and models are publicly available at https://github.com/huwenbin123huwenbin/programming-error-classification. Random seeds are fixed (`random_state=42`). See Section 3.3 for hyperparameters and evaluation protocol details. This research uses publicly available Codeforces data (IRB Approval No. 2026-AI-003, Xijing University).

Ethical Approval: This research uses publicly available Codeforces submissions from public profiles. No private student data was collected. The research was approved by the Institutional Review Board (IRB) of Xijing University (Approval No. 2026-AI-003).

7 Conclusion and Future Directions

Summary of Contributions

This paper presented a extitcontrolled comparative analysis of traditional machine learning (ML) and large language model (LLM) approaches for programming error pattern recognition using *only submission metadata* (no source code access required). Through experiments spanning 5 experimental conditions (3 ML classifiers and 2 LLM configurations) on a curated dataset of 13,360 real-world Codeforces error submissions, we made the following **key contributions**:

(1) *Controlled ML-LLM comparison* for error classification using metadata, filling a critical research gap.
(2) *Comprehensive statistical analysis* with power analysis (13,360 samples, 80% power), effect sizes (Cohen's h), and significance testing (McNemar's test).
(3) *Execution time dominance finding* with ablation validation (8.8 pp accuracy drop, $p < 0.001$).
(4) *Actionable method selection guidelines* balancing accuracy, cost (10–15× lower for ML), latency (2ms vs. 3s), and explainability across six educational contexts.
(5) *Open framework* with publicly released dataset, preprocessing pipeline, and experiment code to facilitate replication.

Key Findings

Finding 1: The Metadata Ceiling. Metadata suffices for three verdicts (CE, TLE, MLE) but not WA↔RE (F1 = 0.52). GB achieves 95.02% accuracy, but 49.6% of RE samples are misclassified as WA, indicating the metadata ceiling. DeepSeek-V3 (76.05%) outperforms Qwen (31.92%) but neither closes the 15.4pp gap to ML.

Finding 2: Execution time dominates (8.8 pp accuracy drop on ablation), confirming runtime behavior as the primary diagnostic signal.

Finding 3: RF and GB show marginal statistical difference ($p = 0.045$) with negligible practical gap (0.5 pp); selection should be guided by deployment constraints.

Finding 4: RE remains hardest (F1 = 0.52), motivating source-code features.

Finding 5: Hybrid ML-LLM approaches—ML for classification, LLM for explanation—offer the best of both paradigms.

Practical Implications

ML enables platform-scale deployment at \$0.01/1k cost and 2ms latency with privacy-preserving analytics, supported by actionable guidelines across six educational contexts (Table 9).

Limitations and Generalizability

Four key limitations: (1) **Population generalizability**—competitive programmers (rating 800–3,500) exhibit different error patterns than novices [12]; (2) **LLM comparison scope and supervised–zero-shot asymmetry**—we tested only DeepSeek-V3 and Qwen2.5:3B under zero-shot conditions; GPT-4, Claude, and Gemini were not evaluated due to API access constraints. Published benchmarks suggest GPT-4 achieves 60–70% on competitive programming code-generation tasks (pass@1) [27], which is broadly consistent with DeepSeek-V3's 76.65% (valid) on our simpler five-way classification. More critically, our ML models are trained on 10,688 labeled samples while our LLMs receive no task-specific training, making this a comparison of *supervised ML* versus *zero-shot LLM*—not the paradigms' full potential. Few-shot prompting or fine-tuning may narrow the gap; this remains untested. Future work should evaluate frontier models (GPT-4, Claude, Gemini) under few-shot conditions to establish a fairer comparison baseline. We designed a 3-shot-per-class prompting protocol (15 examples total, sampled from the training split) but did not execute it due to API cost constraints. Given DeepSeek-V3's zero-shot accuracy of 76.65% (valid), few-shot prompting may narrow the gap to ML, but the magnitude of improvement is uncertain. Recent work on few-shot learning for code tasks suggests gains of 5–15 pp on classification tasks, which would still leave a substantial gap to the 5-feature GB (92.0%). We therefore interpret our zero-shot results as a lower bound on LLM capability rather than an upper bound; (3) **Feature set**—only 7 metadata features were used, and source-code features could improve RE classification (F1 = 0.52); (4) **Theoretical validation**—our framework connects to Hattie–Timperley feedback theory and Bandura's self-efficacy theory, but a longitudinal study would be needed to empirically validate that accurate metadata-based error classification *causes* improved student self-efficacy and learning outcomes.

Future Research Directions

Five research directions: (1) cross-population validation on novice datasets (CS1QA[32], Blackbox) to test whether the metadata ceiling generalizes to students with different error profiles; (2) source-code integration (AST embeddings, code complexity metrics) for improved WA↔RE classification, the educationally most consequential boundary; (3) longitudinal deployment studies to empirically validate the Hattie–Timperley pathway from accurate error classification to improved student self-efficacy; (4) multi-LLM comparison including GPT-4, Claude, Gemini, and LLaMA-3 to determine whether larger models close the gap to ML on this task; (5) real-time deployment evaluation in authentic educational settings, measuring actual learning outcome gains rather than classification accuracy alone.

Closing Remarks

This paper contributes to the ongoing conversation about the role of automated analysis in computing education by providing the first precise decomposition of what submission metadata can and cannot reveal about programming error type. Our central finding—that the WA↔RE boundary defines the precise frontier of metadata sufficiency—has both practical and theoretical implications. Practically, it tells developers of educational platforms exactly where to draw the line between fully automated classification and source-code-requiring analysis. Theoretically, it refines our understanding of the information structure of programming assessment: execution metadata encodes resource-bounded verdict information reliably, but the deeper semantic distinction between a logic error and a runtime hazard requires richer representational signals.

Connecting to Hattie and Timperley's feedback framework, our findings imply that metadata-based automated feedback can realistically operate at the task level for three of five verdict types, supporting process-level feedback through accurate error attribution. Wiliam and Leahy [54] further establishedhed that effective formative assessment requires students to understand where they are relative to the learning goal, not merely whether their submission was correct; accurate error classification is a prerequisite for this level of feedback granularity. ealistically operate at the task level for three of five verdict types, supporting process-level feedback through accurate error attribution. For the WA$\leftrightarrow$RE boundary—precisely where the student must revise algorithmic strategy versus check boundary conditions—the system should flag uncertainty and invite human review, rather than committing to a specific error attribution. This calibrated uncertainty is itself pedagogically valuable: it signals to the student that the error requires deliberate analysis rather than mechanical correction.

Hybrid deployment—ML for fast, cheap verdict classification; LLMs conditionally invoked for explanation generation at the WA$\leftrightarrow$RE boundary—represents the most cost-effective path toward intelligent programming education support, balancing automated scalability with the depth of human expert guidance.

Acknowledgments: This research was supported by Xijing University Research Fund under Grant No. XJ-2025-001.

Data Ethics and Privacy

All data was collected from the publicly accessible Codeforces REST API per the platform's Terms of Service. Usernames were anonymized. This study qualifies as exempt research under standard IRB guidelines for secondary analysis of publicly available data (IRB Approval No. 2026-AI-003).

Data and Code Availability

All data and code are available at
https://github.com/huwenbin123huwenbin/programming-error-classification.

References

[1] A. Ahadi, R. Lister, H. Haapala, and A. Vihavainen. 2015. Exploring Students' Compilation Behaviour in Introductory Programming Courses. In *Proceedings of the 15th Koli Calling Conference on Computing Education Research*. 3–12. doi:10.1145/2828959.2828963

[2] Miltiadis Allamanis, Marc Brockschmidt, and Mahmoud Khademi. 2018. Learning to represent programs with graphs. In *International Conference on Learning Representations (ICLR)*.

[3] A. Alnahhas and N. Mourtada. 2020. Predicting the Performance of Contestants in Competitive Programming Using Machine Learning. *Olympiads in Informatics* 14 (2020), 7–26. doi:10.15388/ioi.2020.01

[4] Amjad Altadmri and Neil C. C. Brown. 2015. What are the most common errors in novice programs?. In *Proceedings of the 46th ACM Technical Symposium on Computer Science Education*. ACM, 102–107.

[5] T. Baker, G. E. Smith, and N. Goldsmith. 2009. Educational data mining: An introduction to the special issue. *Journal of Educational Data Mining* 1, 1 (2009), 1–5.

[6] Albert Bandura. 1997. *Self-efficacy: The Exercise of Control*. Freeman.

[7] H. J. Barbosa Rocha, P. Cabral de A. Tedesco, and E. de Barros Costa. 2022. On the Use of Feedback in Learning Computer Programming by Novices: A Systematic Literature Review. *Informatics in Education* 22, 1 (2022), 125–152. doi:10.15388/infedu.2023.09

[8] Shraddha Barke, Michael B. James, and Nadia Polikarpova. 2023. Grounded Copilot: How Programmers Interact with Code-Generating Models. In *Proceedings of the ACM on Programming Languages*, Vol. 7. ACM, 85:1–85:29. doi:10.1145/3586030

[9] Brett A. Becker, Paul Denny, James Finnie-Ansley, et al. 2023. Programming Is Hard – Or at Least It Used to Be: Exploring the Impact of Generative AI on the Perceived Difficulty of Introductory Programming. In *Proceedings of the 2023 ACM Conference on Innovation and Technology in Computer Science Education (ITiCSE '23)*. ACM, to appear. doi:10.1145/3545945.3569759

[10] J. Bennedsen and M. E. Caspersen. 2007. Failure rates in introductory programming. *ACM SIGCSE Bulletin* 39, 2 (2007), 32–36. doi:10.1145/1272848.1272879

[11] Leo Breiman. 2001. Random forests. *Machine Learning* 45, 1 (2001), 5–32. doi:10.1023/A:1010933404324

[12] Neil C. C. Brown and Michael Kölling. 2014. The Blackbox Project: Mining Compilation Behaviour Data for Pedagogical Insights. In *Proceedings of the 45th ACM Technical Symposium on Computer Science Education (SIGCSE '14)*. ACM, New York, NY, 113–118. doi:10.1145/2538862.2538924

[13] Teresa Busjahn, Roman Bednarik, and Andrew Begel. 2015. Eye Movements in Code Reading: Relaxing the Linear Order. In *2015 IEEE International Conference on Program Comprehension*. IEEE, 255–264. doi:10.1109/icpc.2015.36

[14] Jacob Cohen. 1988. *Statistical Power Analysis for the Behavioral Sciences* (2nd ed.). Lawrence Erlbaum Associates.

[15] DeepSeek-AI. 2024. DeepSeek-V3: A Technical Report. arXiv preprint arXiv:2412.19437. doi:10.48550/arXiv.2412.19437

[16] Paul Denny, Juho Leinonen, and James Prather. 2024. Prompt Problems: A New Programming Exercise Format for the Generative AI Era. *Proceedings of the 2024 ACM Conference on Innovation and Technology in Computer Science Education (ITiCSE '24)* (2024), to appear. doi:10.1145/3626252.3630909

[17] T. Effenberger and R. Pelánek. 2021. Validity and Reliability of Student Models for Problem-Solving Activities. In *Proceedings of the 11th International Conference on Learning Analytics and Knowledge*. 146–156. doi:10.1145/3448139.3448140

[18] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Peng, Bing Qin, Ting Liu, and Ming Zhou. 2020. CodeBERT: A pre-trained model for programming and natural languages. In *Findings of the Association for Computational Linguistics: EMNLP 2020*. 9161–9168.

[19] J. Finnie-Ansley, P. Denny, B. A. Becker, A. Luxton-Reilly, and J. Prather. 2022. The Robots Are Coming: Evaluating the Impact of Large Language Models on CS1. In *Proceedings of the 24th Australasian Computing Education Conference*. 10–19. doi:10.1145/3511861.3511863

[20] Jerome H Friedman. 2001. Greedy function approximation: A gradient boosting machine. *Annals of Statistics* 29, 5 (2001), 1189–1232.

[21] Jamie Gorson and Eleanor O'Rourke. 2020. Why Do CS1 Students Think They're Bad at Programming?. In *Proceedings of the 2020 ACM Conference on Innovation and Technology in Computer Science Education (ITiCSE '20)*. ACM, 278–284. doi:10.1145/3372782.340827

[22] John Hattie and Helen Timperley. 2007. The Power of Feedback. *Review of Educational Research* 77, 1 (2007), 81–112. doi:10.3102/003465430298487

[23] David W Hosmer, Stanley Lemeshow, and Rodney X Sturdivant. 2013. Logistic regression. *Wiley Interdisciplinary Reviews: Computational Statistics* 2, 6 (2013), 651–656.

[24] Matthew C. Jadud. 2006. Exploring Two Sets of Barriers to Programming. In *Proceedings of the 2006 International Workshop on Computing Education Research (ICER '06)*. Association for Computing Machinery, New York, NY, USA, 89–100.

[25] M. C. Jadud. 2006. Methods and tools for exploring novice compilation behaviour. *Computer Science Education* 16, 3 (2006), 193–217. doi:10.1145/1151588.1151600

[26] Sandeep M. Jayaprakash, Erik W. Moody, E. J. M. Lauria, et al. 2014. Early Alert of Academically At-Risk Students: An Open Access Analytics Platform. *Journal of Learning Analytics* 1, 3 (2014), 6–21. doi:10.18608/jla.2014.11.3

[27] Mingjie Jin, Erfan Allahgholi, Luca Pirrone, Giovanni Bravi, Graziano Riccia, Harshit Shrivastava, Alexey Svyatkovskiy, Ziyang Zhuang, Wasi Chen, Yonghui Wu, et al. 2024. CrossCodeBench: A Benchmark for Cross-Code Understanding and Generation in Software Development. *arXiv preprint*

arXiv:2405.15104 (2024).

[28] M. Kazemitabaar, J. Chow, C. Ma, B. J. Ericson, D. Weintrop, and T. Grossman. 2024. The Impact of Generative AI on Student Learning in CS1. In *Proceedings of the 2024 ACM Conference on International Computing Education Research*. 123–135. doi:10.1145/3626252.3630909

[29] Hieke Keuning, Johan Jeuring, and Bastiaan Heeren. 2017. A Systematic Literature Review of Automated Feedback Generation for Programming Exercises. *ACM Transactions on Computing Education* 17, 4 (2017), 13:1–13:44. doi:10.1145/3105759

[30] Paul A. Kirschner, John Sweller, Femke Kirschner, and Jacobo Zambrano R. 2018. From Cognitive Load Theory to Collaborative Cognitive Load Theory. *International Journal of Computer-Supported Collaborative Learning* 13 (2018), 213–233. doi:10.1007/s11412-018-9277-y

[31] Peter A. Lachenbruch. 1998. Power and Sample Size Requirements for Logistic Regression. *Statist. Med.* 17 (1998), 285–297.

[32] C. Lee, D. Kim, and H. Kim. 2022. CS1QA: A Dataset for Assisting Code-Based Question Answering in an Introductory Programming Course. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics*. 321–328. doi:10.18653/v1/2022.naacl-main.23

[33] J. S. Y. Lee, F. Liu, and T. Cai. 2024. Code Debugging with LLM-Generated Explanations of Programming Error Messages. In *2024 IEEE 13th International Conference on Engineering Education (ICEED)*. 1–6. doi:10.1109/iceed62316.2024.10923833

[34] C. Leerentveld, L. Nienaber, T. van der Bij, et al. 2024. Not the Silver Bullet: LLM-enhanced Programming Error Messages are Ineffective in Practice. In *Proceedings of the 2024 ACM Conference on International Computing Education Research (ICER)*. doi:10.1145/3689535.3689554

[35] Juho Leinonen, Arto Hellas, and Sami Sarsa. 2023. Using Large Language Models to Enhance Programming Error Messages. In *Proceedings of the 2023 ACM Conference on International Computing Education Research*. ACM, 563–573. doi:10.1145/3545945.3569770

[36] Andrew Luxton-Reilly, Simon, Ali A. C. Al-Zaidi, et al. 2018. Introductory Programming: A Systematic Review. *Comput. Surveys* 51, 3 (2018), 1–35. doi:10.1145/3293881.329577

[37] L. McCauley, N. Mazur, S. Fitzgerald, E. Murphy-Hill, T. Quincey, and C. Rost. 2008. An evidence-based systematic review of novice programmer mistakes. *ACM SIGCSE Bulletin* 40, 3 (2008), 131–152. doi:10.1145/1356322.1356323

[38] Quinn McNemar. 1947. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika* 12, 2 (1947), 153–157.

[39] Mike Mirzayanov and Oksana Pavlova. 2020. Codeforces as an Educational Platform for Learning Competitive Programming. In *Informatics in Education*, Vol. 19. Vilnius University Institute of Mathematics and Informatics, 231–250. doi:10.15388/ioi.2020.10

[40] Rebecca Morris, Thomas Perry, and Louise Wardle. 2021. Formative Assessment and Feedback for Learning in Higher Education: A Systematic Review. *Review of Education* 9, 2 (2021), e3292. doi:10.1002/rev3.3292

[41] Ernesto Panadero. 2017. A Review of Self-Regulated Learning: Six Models and Four Directions for Research. *Frontiers in Psychology* 8 (2017), 1–28. doi:10.3389/fpsyg.2017.004

[42] A. Pears, M. Seidman, L. Malmi, L. Mannila, E. Adams, J. Bennedsen, M. Devlin, and J. Paterson. 2007. A survey of literature on the teaching of introductory programming. *ACM SIGCSE Bulletin* 39, 4 (2007), 204–223. doi:10.1145/1345443.1345441

[43] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. 2011. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research* 12 (2011), 2825–2830.

[44] Chris Piech, Jonathan Spencer, and Jonathan Huang. 2015. Deep Knowledge Tracing. *arXiv preprint arXiv:1506.05908* (2015). arXiv:1506.05908 [cs.AI] doi:10.48550/arXiv.1506.05908

[45] James Prather, Paul Denny, and Juho Leinonen. 2023. The Robots Are Here: Navigating the Generative AI Revolution in Computing Education. In *Proceedings of the 2023 ACM Conference on International Computing Education Research*. ACM, 108–109. doi:10.1145/3623762.3633499

[46] J. Prather, B. Reeves, P. Denny, B. A. Becker, J. Leinonen, A. Luxton-Reilly, G. Powell, J. Finnie-Ansley, and E. A. Santos. 2024. Code of the Dead: Analyzing LLM-Generated Code in CS1 Assignments. In *Proceedings of the 55th ACM Technical Symposium on Computer Science Education*. 1042–1048. doi:10.1145/3649165.3690125

[47] N. Raihan, M. Latif Siddiq, J. C. S. Santos, and R. Hossen. 2025. Large Language Models in Computer Science Education: A Systematic Literature Review. In *Proceedings of the 56th ACM Technical Symposium on Computer Science Education*. 937–943. doi:10.1145/3641554.3701863

[48] C. Romero and S. Ventura. 2013. Data mining in education. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery* 3, 1 (2013), 12–27. doi:10.1002/widm.1075

[49] Sami Sarsa, Paul Denny, and Arto Hellas. 2022. Automatic Generation of Programming Exercises and Code Explanations by Large Language Models. *ACM Transactions on Computing Education* 22, 4 (2022), 1–29. doi:10.1145/3501385.3543957

[50] J. C. Spohrer and E. Soloway. 1986. Novice mistakes: Are there just a few fundamental bugs? *Commun. ACM* 29, 7 (1986), 627–643. doi:10.1145/6138.6145

[51] John Sweller. 2011. Cognitive Load Theory. In *Psychology of Learning and Motivation*, Jose P. Mestre and Brian H. Ross (Eds.). Vol. 55. Academic Press, 37–76. doi:10.1016/B978-0-12-387691-1.00002-8

[52] Katrien Verbert, Erik Duval, and Joris Klerkx. 2013. Learning Analytics Dashboard Applications. *American Behavioral Scientist* 57, 10 (2013), 1500–1523. doi:10.1177/0002764213479363

[53] C. Watson and F. W. Li. 2014. Failure rates in introductory programming revisited. *ACM Transactions on Computing Education* 18, 2 (2014), 1–21. doi:10.1145/2591708.2591749

[54] Dylan Wiliam. 2011. What is Assessment for Learning? *Studies In Educational Evaluation* 37 (2011), 3–14. doi:10.1016/j.stueduc.2011.03.001

[55] Leon E. Winslow. 1996. Programming Pedagogy: A Psychological Overview. *ACM SIGCSE Bulletin* 28, 3 (1996), 17–22. doi:10.1145/234867.234872

[56] Aman Yadav, Chris Mayfield, and Sukanya Kannan Moudgalya. 2021. Collaborative Learning, Self-Efficacy, and Student Performance in CS1 POGIL. In *Proceedings of the 52nd ACM Technical Symposium on Computer Science Education*. ACM, 775–781. doi:10.1145/3408877.3432373

[57] Jialu Zhang and De Li. 2022. Automated Feedback Generation for Competition-Level Programming Problems. In *Proceedings of the 2022 ACM Conference on Learning@ Scale (L@S '22)*. ACM, 139–150. doi:10.1145/3551349.356042